IC100: Introduction to Computing

Course Logistics

Instructor Details

Dr. Sk Subidh Ali

Assistant Professor

Department of Computer Science and Engineering

Office: 411B,

Engineering Building 1 (ED1)

Email: subidh@iitbhilai.ac.in

Teaching Assistants (TA)

Sudev Kumar Padhi

Chief TA

20 More TAs will assist you through out course

The Course

- The course introduces you to programming using the C programming language
- No prior exposure to programming is needed
- You will be evaluated on both theoretical and practical aspects of programming, using written exams and labs

Why am I Doing This Course?

- Every discipline uses computing: All branches of engineering, sciences, design and arts.
- Understand how computers work
- Write your own programs
  - Automate boring repetitive stuff!

Lectures

- Lectures
  - Mon, 08:30 am – 09:30 am , L209
  - Wed, 9:30 am 10:30 pm, L209

Tutorials

- There will be tutorials every Friday
  - 9:30 AM - 10:30 PM
- There will be 4 batches for tutorials
  - Batch I, Batch II, Batch III, Batch IV (L201, L202, L205,L206, L208)
- You can ask questions and clarify doubts regarding lecture material
- Examples illustrating lecture material will be covered
- Quizzes will be held during the last 15 minutes of every tutorial sessions

Labs

- Lab is divided into 3 Batches:
  - Batch I, Batch II, Batch III,
- Schedule: 2:30 PM – 5:30 PM
  - Batch I : Monday
  - Batch II : Tuesday
  - Batch IV : Wednesday
  -
- Location:
  - Room no 319 -ED 1 and 303 ED 2

- Labs start From 5th August
- First week labs are meant to familiarize you with the lab workflow

Labs

- Sunday: Could be used to make up for lab days lost due to holidays.
- There will be Teaching Assistants (TAs) to help in the labs.
- In each lab, you will be given a few problems to solve.
  - Students must work on their own.
  - Discussion is allowed, but sharing of code in any form is NOT permitted.

Lab Assignments

- Lab assignment will be posted on the day of the lab, at 2:30 PM
- It has to be submitted by 5:20 PM
- There will additional question carrying bonus marks

Weightage (Theory)

- Quizzes: 7%
  - Conducted in lecture and tutorial hours
- Mid Sem: 25%
- End Sem : 35%
- Lab : 25%
- Homework: 3%
- Classroom Assesment: 5%

Weightage (Lab)

- Labs:
  - Weightage of later labs may be more. (First lab: 0 weight)
  - No make-up lab for absentees.
- Lab exams:

  MidSem:

  EndSem :

Semester Workload

- Every week
  - 2 hours lectures
  - 1 hour tutorial
  - 3 hours lab work
- Semester
  - 30 Quizzes
  - 2 lab exams
  - 2 theory exams

Copying

- Copy at your own risk
  - in any component (lab/quiz/exams/lab exams)
  - If you are caught, you get penalised on grade (most likely F)
  - Will not be allowed to drop the course
  - Case reported to DoAA
  - No warning or second chance
  - All parties involved in copying will be held equally responsible. Copying from internet is penalized equally
  - Policy may change on need basis

Copying

- Read-protect your directories so that others cannot copy from your directory
- Do not share your codes with friends
- Do not leave printouts, notes etc. containing your code unattended

Course Websites

Course web site

I will give a piazza link and course website

Course Materials

- Every student must register this course in Piazza.
- All course materials, including lectures, exam solutions, quiz solutions etc., will be posted on Piazza and subsequently on course web site.
- Use piazza for interaction
  - Allows instructor, tutors and your classmates to answer any issues

Textbooks

- There are several books available on C programming language
  - C in Depth by Deepali Srivastava
  - The C Programming Language by Kernighan and Ritchie, Prentice-Hall India
  - Programming in ANSI C by Balaguruswamy
  - Any other standard book on C would also be good

- It is recommended that you have a book and refer to it throughout the semester and beyond

E-Mailing

- Please make sure you mention your section and roll number in emails
  - While Submission don’t forget to metion name, roll number and batch of the lab.
  - Prefer using discussion feature of Piazza

Running your Code

- Have your own laptop
- For windows- WSL terminal
- For linux- gcc

Today’s Class

- What is programming?
- What is the programming workflow?
- What do simple C programs look like?

What is a Computer ?





A computer is a general purpose device that can be programmed to process information, and yield meaningful results.

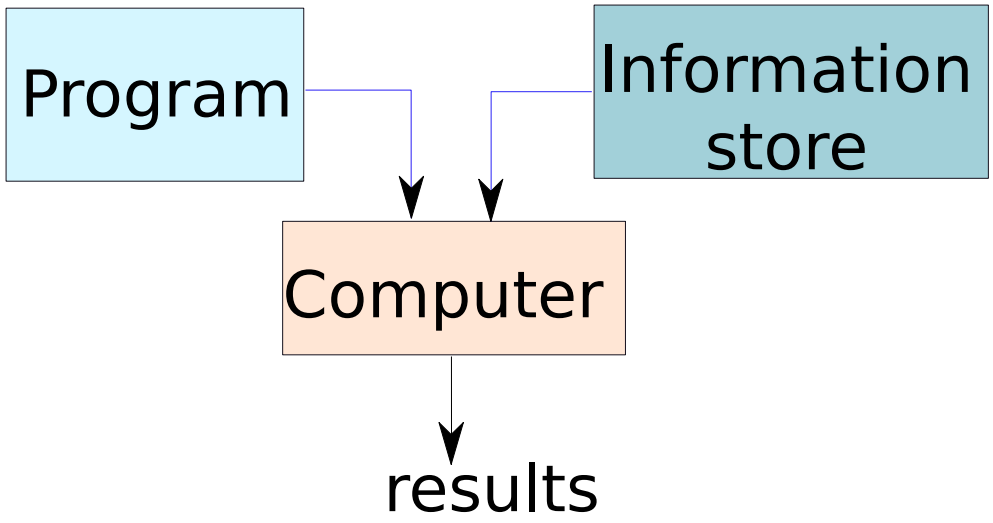




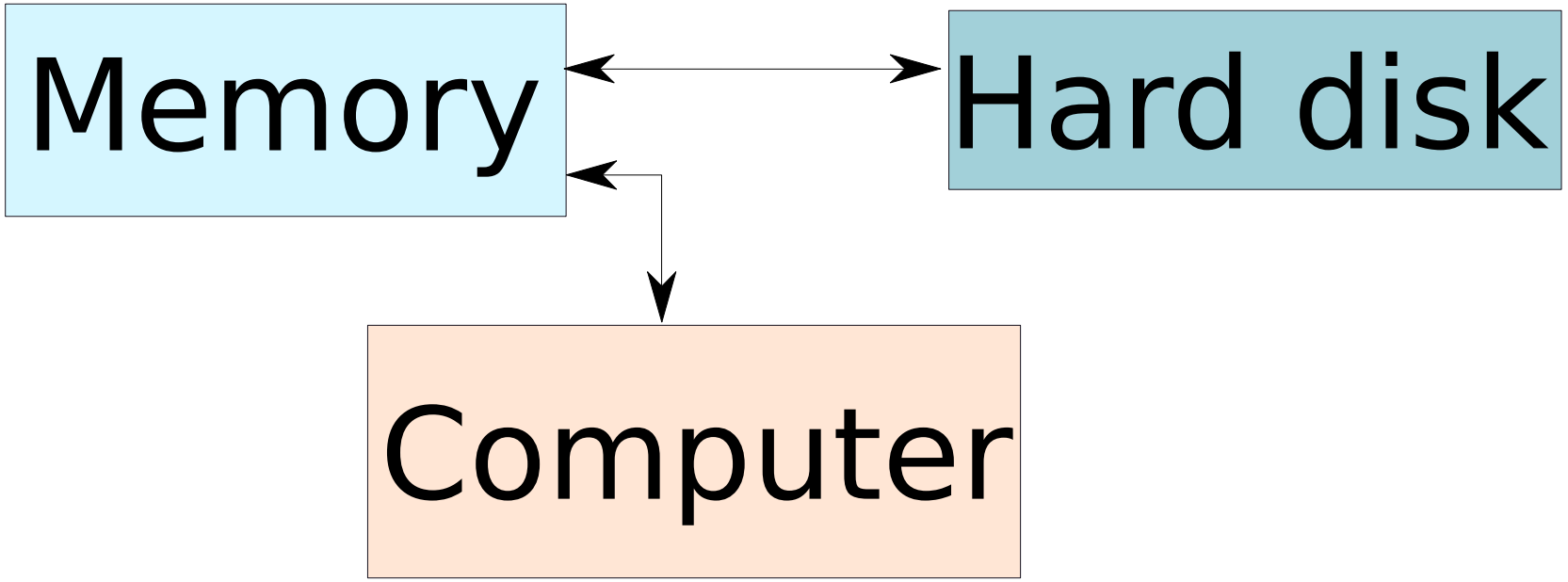
What is a Computer Program?

- **Computer:** Device which can be instructed to carry out arithmetic and logical operations automatically
  - Must take inputs
  - Must store inputs
  - Must process input
  - Must give outputs
- How would the Computer process the inputs?
  - Needs precise instructions PROGRAM
- We will learn programming in this course

How does it work ?

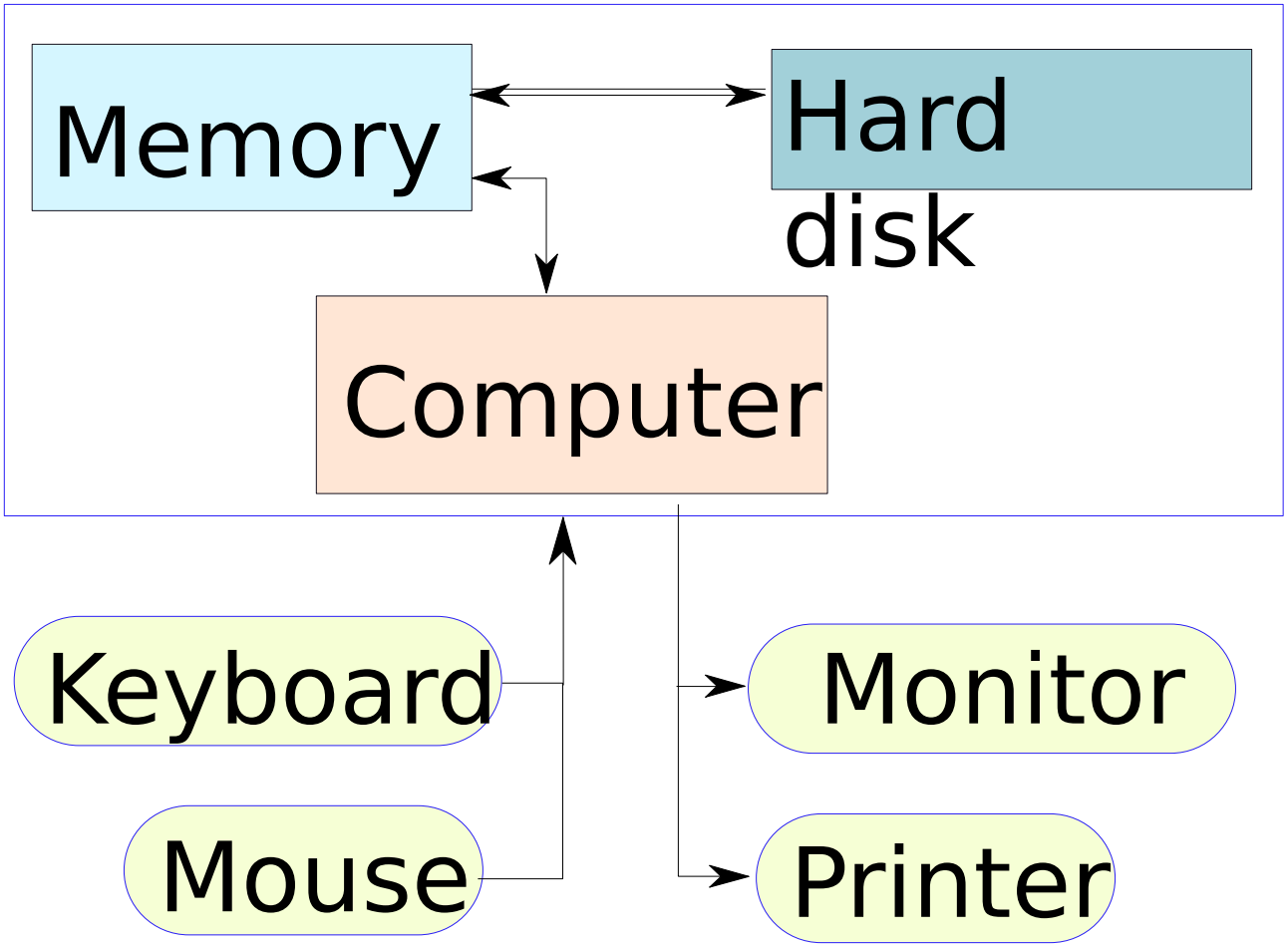


- Program – List of instructions given to the computer
- Information store – data, images, files, videos
- Computer – Process the information store according to the instructions in the program



- Memory – Stores programs and data. Gets destroyed when the computer is powered off
- Hard disk – stores programs/data permanently

Let us make it a full system ...



Process of Programming: Modelling

- Define and model the problem.
  - In real-life this is important and complicated.
- Modelling the section assignment process
  - Explicit Input list of people with attributes
    - Roll number
    - Name
    - Section preference
  - Implicit inputs modelling
    - an equality constraint
    - a comprehensiveness constraint
    - Number of sections
  - Output
    - 12 equal* lists, no student left without a section

Process of Programming: Algorithm

- Obtain a logical solution to your problem.
- A logical solution is a finite and clear step-by-step procedure to solve your problem.
- Also called an Algorithm (or *recipe*).

Algorithms in Ordinary Life? (Recipes!)

- An algorithm is a familiar concept: cooking recipes are almost algorithms!

- Ingredients

1. 1 liter (33 oz) ice cream (any flavor).
2. Crushed cereal, such as corn flakes, frosted flakes, cinnamon squares, or puffed rice.
3. Flour (a small bowl of it, approx 1/2 cup).
4. Oil (use an unflavoured oil that has a high heat point).
5. 2 eggs (beaten in a bowl large enough for dipping).
6. Cinnamon and/or sugar (optional).

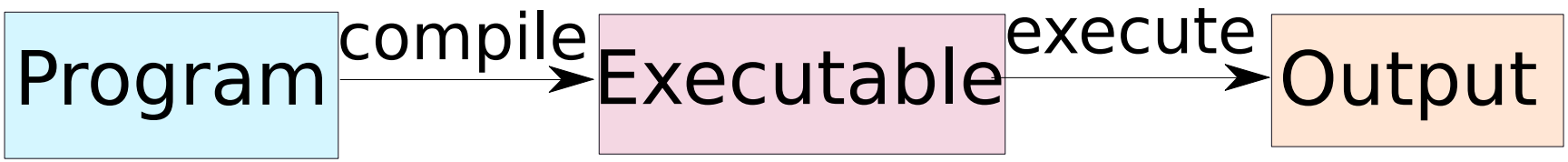
- Instructions

1. Prepare the two baking sheets by lining with a silicon liner or parchment paper. Then place the sheets in the freezer for half an hour prior to making the ice cream balls.
2. Scoop symmetrical balls of ice cream. Try to make each scoop about as large as your fist. Make as many scoops as will fit on the baking sheets.
3. Harden the scooped ice cream balls in the freezer.
4. Set out the bowls for dipping. Place a bowl of flour, a bowl of beaten egg and a bowl of crushed cereal or fine cookie/cracker crumbs in the workspace, in a formation that makes it easy to dip in order.
5. Coat the ice cream.
6. Place the ice cream balls back on the baking sheets, then back in the freezer.
7. Fry the coated ice cream balls. Heat up the oil until it shimmers - approx 185C.
8. Serve the ice cream balls.

Source: Wikihow: Fried Ice-cream

not quite precise enough for a computer!

How to Instruct a Computer ?



- Write a program in a high level language – C, C++, Java
- **Compile** it into a format that the computer understands
- Execute the program

What Can a Computer Understand ?

- Computer can clearly NOT understand instructions of the form
  - Multiply two matrices
  - Compute the determinant of a matrix
  - Find the shortest path between Mumbai and Delhi
- They understand :
  - Add $a + b$ to get c
  - Multiply $a + b$ to get c

The Language of Instructions

- Humans can understand
  - Complicated sentences
    - English, French, Spanish
- Computers can understand
  - Very simple instructions

The semantics of all the instructions supported by a processor is known as its instruction set architecture (ISA). This includes the semantics of the instructions themselves, along with their operands, and interfaces with peripheral devices.

Fundamentals: Binary Numbers

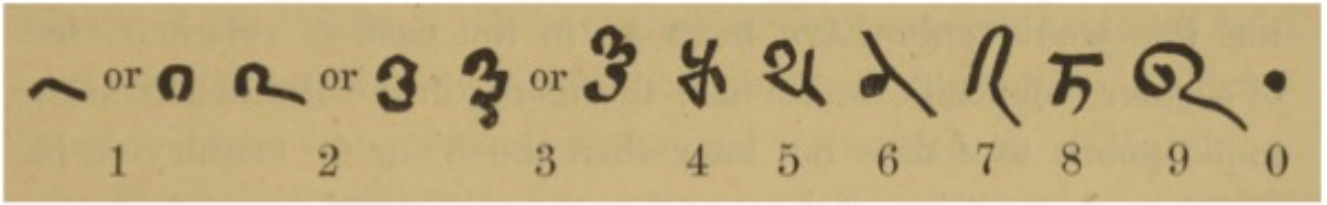
Representing Positive Integers

* Ancient Roman System

Issues :

* There was no notion of 0
* Very difficult to represent large numbers
* Addition, and subtraction (very difficult)

Indian System



Bakshali numerals, 7th century AD

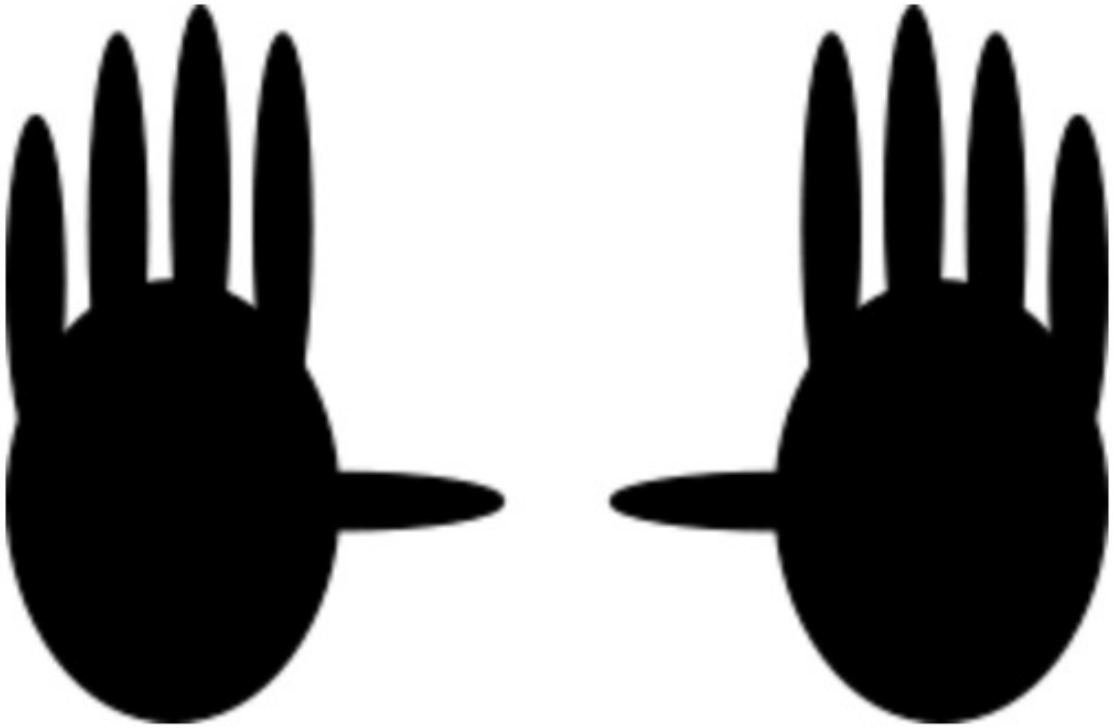
* Uses the place value system

$$5301 = 5 * 10^3 + 3 * 10^2 + 0 * 10^1 + 1*10^0$$

Example in base 10

Number Systems in Other Bases

* Why do we use base 10 ?
  * because ...



What if we had a world in which ...

* People had only two fingers.



Representing Numbers

$1023_{10} = 1 * 10^3 + 0 * 10^2 + 2 * 10^1 + 3 * 10^0$

Decimal

base

Binary

$17_{10} = 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 = 10001_2$

Base = 2

Binary Number System

* They would use a number system with base 2.

Number in decimal	Number in binary
5	101
100	1100100
500	111110100
1024	10000000000

MSB and LSB

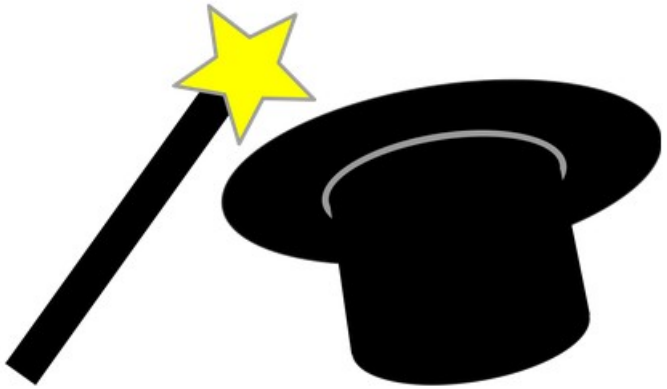
* MSB (Most Significant Bit) The leftmost bit of a binary number. E.g., MSB of 1110 is 1
* LSB (Least Significant Bit) The rightmost bit of a binary number. E.g., LSB of 1110 is 0

Hexadecimal and Octal Numbers

* Hexadecimal numbers
  * Base 16 numbers – 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
  * Start with 0x
* Octal Numbers
  * Base 8 numbers – 0,1,2,3,4,5,6,7
  * Start with 0

Examples

Decimal	Binary	Octal	Hexadecimal
9	1001	0 11	0x 9
12	1100	0 14	0x C
17	10001	0 21	0x 11
28	11100	0 34	0x 1C



Convert 110010111 to the octal format : = 0612

Convert 111000101111 to the hex format : = 0xC2F

Bits and Bytes

- Computers do not understand natural human languages, nor programming languages
- They only understand the language of bits

Bit	0 or 1
Byte	8 bits
Word	4 bytes
kiloByte	1024 bytes
megaByte	10^6 bytes

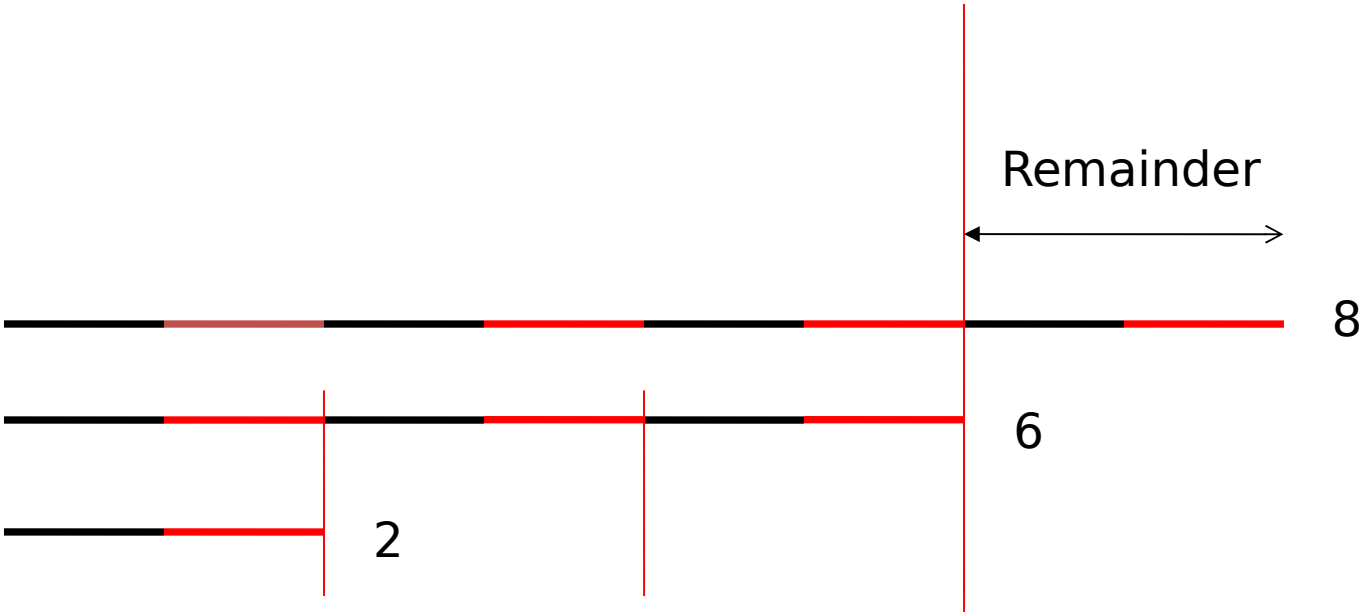
GCD

- An algorithm to find the greatest common divisor of two positive integers m and n, m ≥ n.
- The ugly way
  1. Take the smaller number p.
  2. For each number k, p ≥ k ≥ 1, in descending order, do the following.
     a) If k divides m and n, then k is the gcd of m and n

GCD

- This will compute gcd correctly, but is VERY slow (think about large numbers m and n=m-1).
- There is a faster way…

GCD – the beautiful way

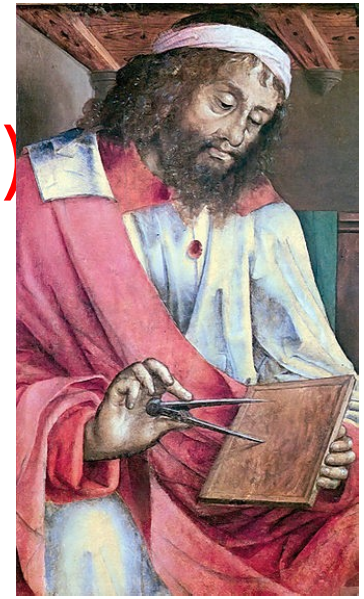


Gcd(8, 6) = 2.

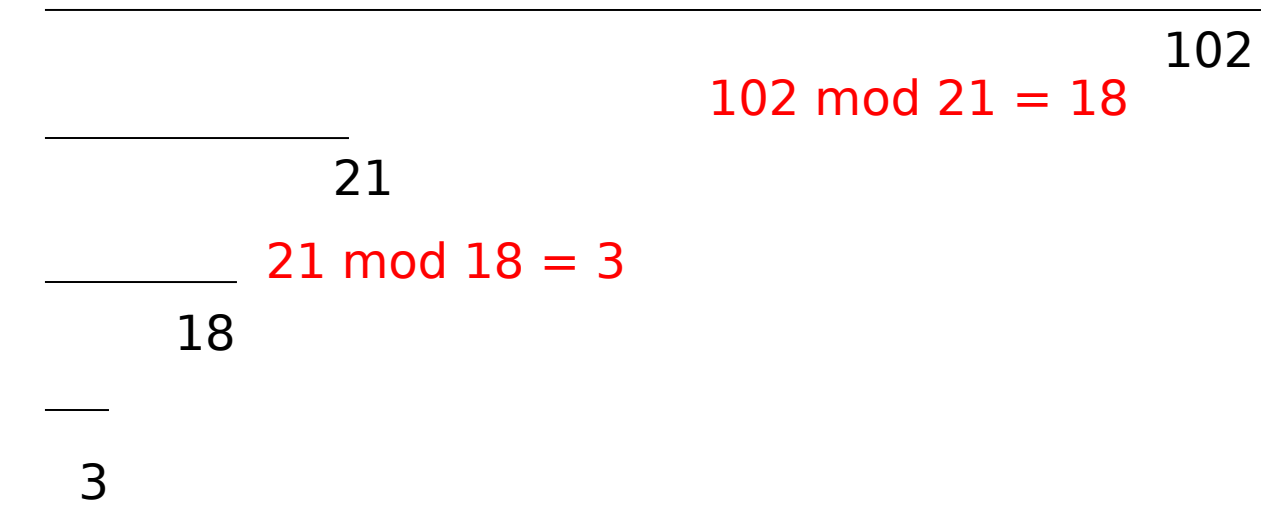
Euclidean algorithm

- Suppose a > b. Then the gcd of a and b is the same as the gcd of b and the remainder of a when divided by b.

  gcd(a,b) = gcd (b, a % b)



The Euclidean algorithm



Gcd (102, 21) = 3

GCD Algorithm

Data: Integers m and n

if n> m then interchange m and n

while (n ≠ 0) do

    g = m mod n

    m = n

    n = g

    end

    return m

Data: m = 80 and n = 30

Iteration 1:
g = 20, m = 30, n = 20

Iteration 2:
G = 10, m = 20, n = 10

Iteration 3:
G = 0, m = 10, n = 0

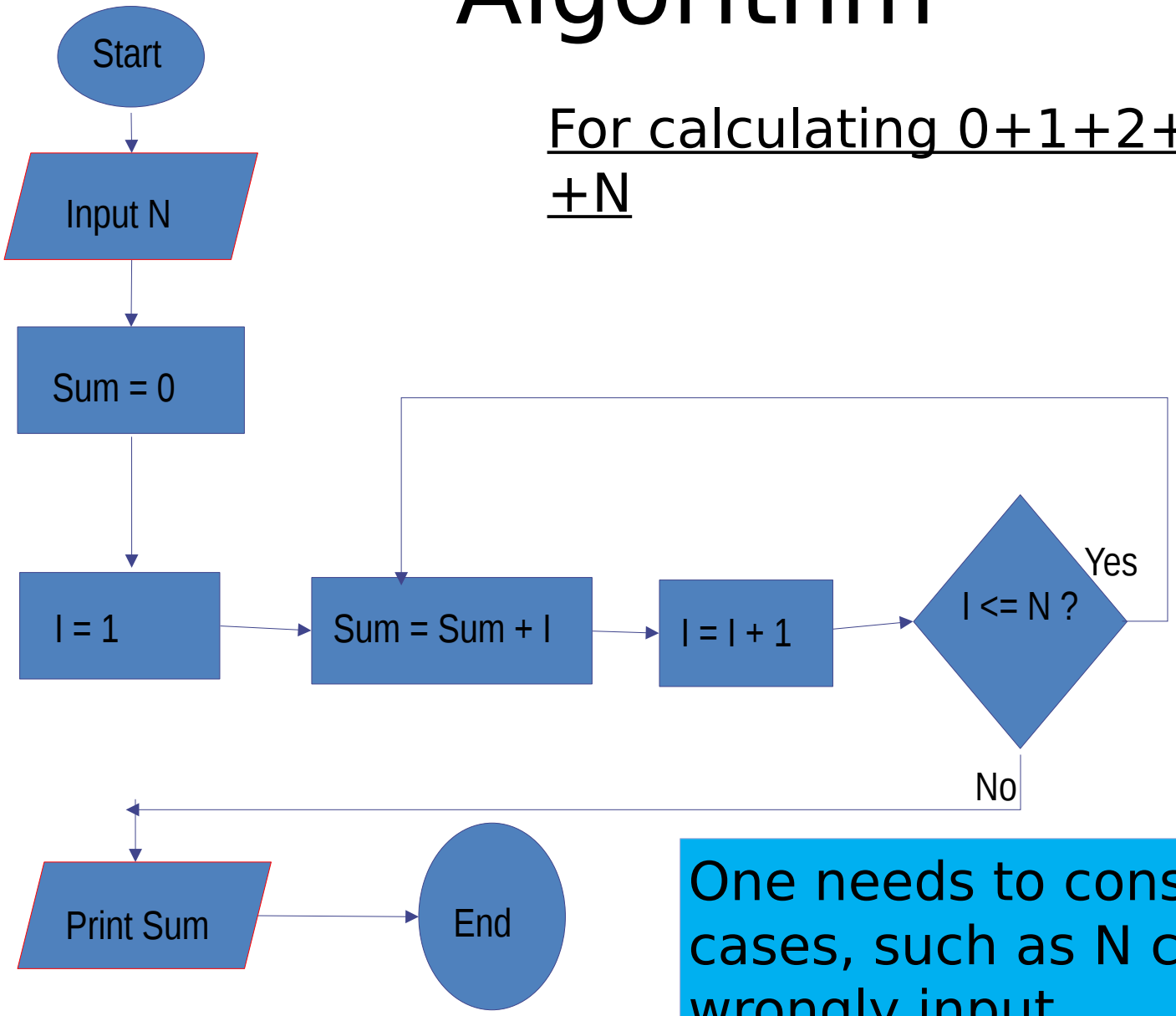
Terminates
GCD (80, 30) = 10

Flow Chart

- We can visualize an algorithm using a Flowchart.

- Very important step in the programming process.

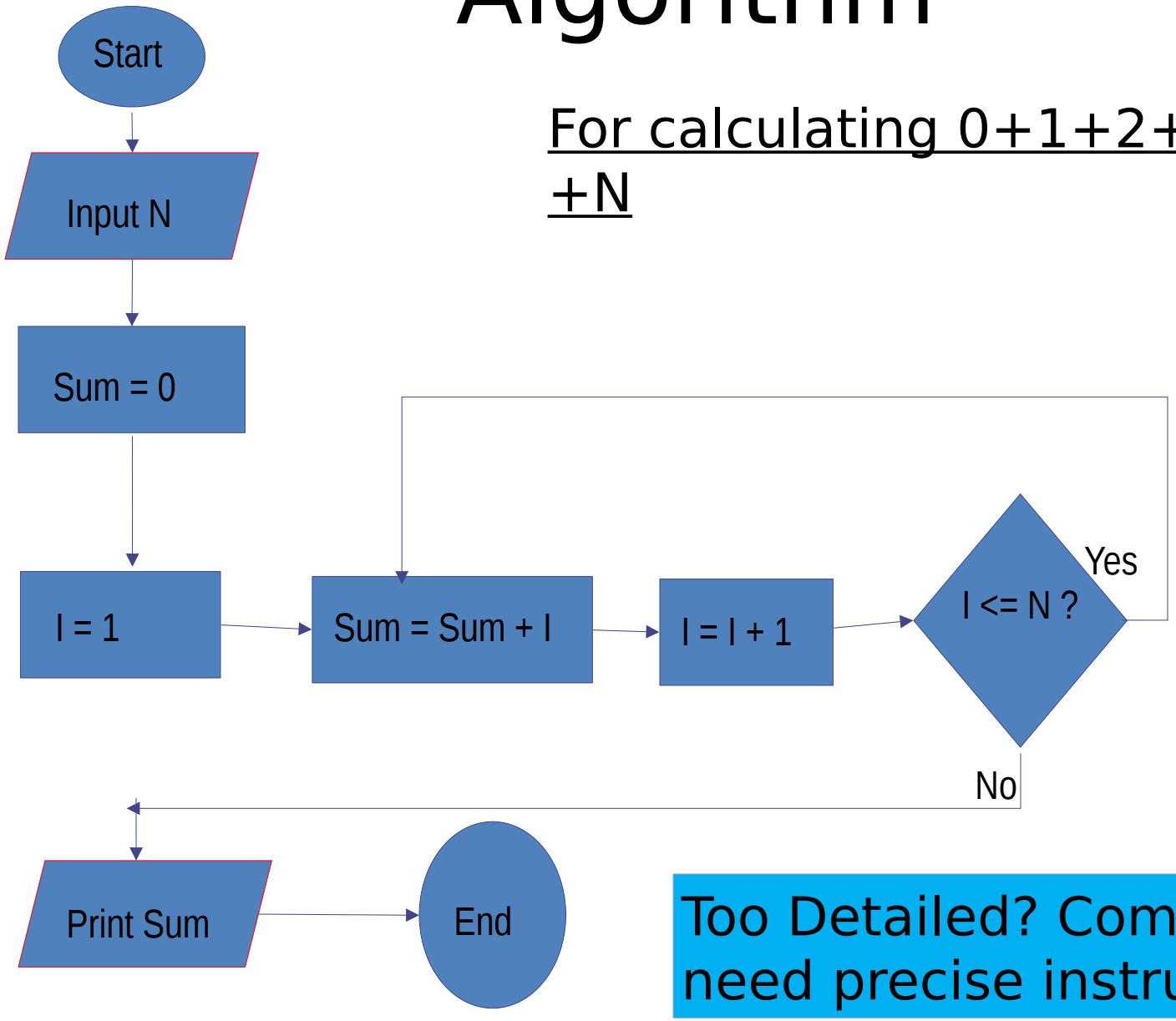
Flowchart to Depict Algorithm



For calculating 0+1+2+…+N

One needs to consider all cases, such as N could be wrongly input

Flowchart to Depict Algorithm

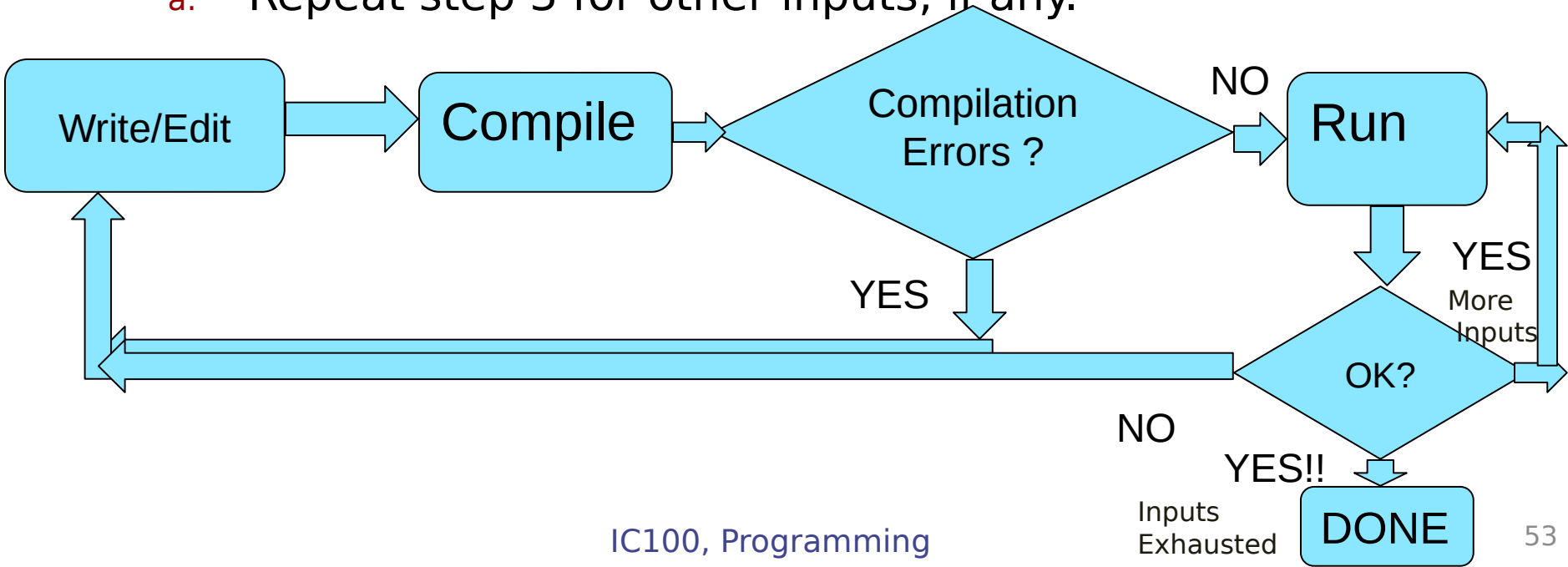


For calculating 0+1+2+…+N

Too Detailed? Computers need precise instructions

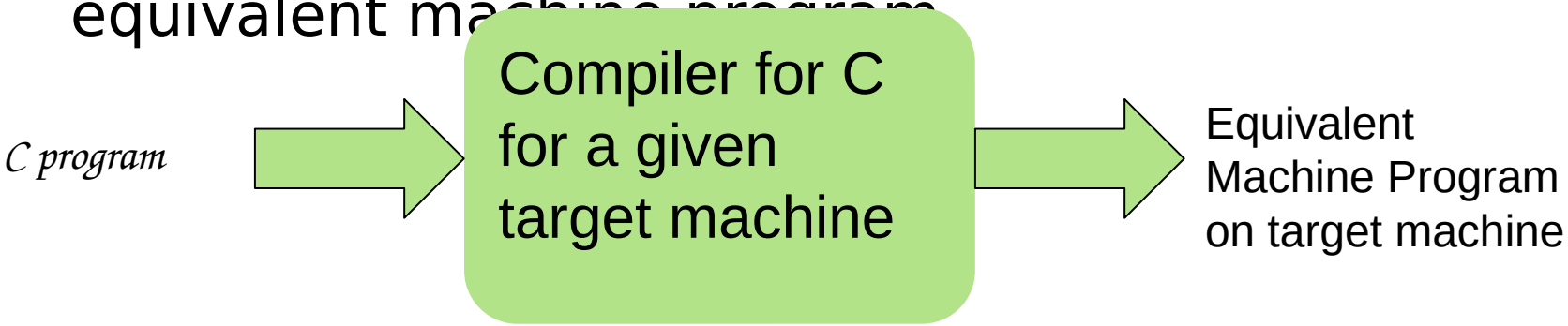
The Programming Cycle

1. Write your program or edit (i.e., change or modify) your program.
2. Compile your program. If compilation fails, return to editing step.
3. Run your program on an input. If output is not correct, return to editing step.
   a. Repeat step 3 for other inputs, if any.



Why program in high level languages like C

- Writing programs in machine language is long, tedious and error-prone.
- They are also not portable—meaning program written for one machine may not work on another machine.
- Compilers work as a bridge.
- Take as input a C program and produce an equivalent machine program



IDE for Edit-Compile-Run cycle

- In this course we will be using gedit or simple text editor in ubuntu to write the programs.
- First login to the system.
- The labs in the first week will introduce you to the system in more detail.

Simple Program

- In the first week, we practice the simplest C programs.

```
# include <stdio.h>
int main ()  {
    printf("Welcome to IC100");
    return 0;
}
```

The program prints the message “Welcome to ESC101”

Program Components

```
# include <stdio.h>
int main ()
{
    printf("Welcome to IC100");
    return 0;
}
```

1. This tells the C compiler to include the standard input output library.

2. Include this line routinely as the first line of your C file.

printf is the *function called* to output from a C program. To print a string, enclose it in " " and it gets printed.

"return" returns the control to the caller (program finishes in this case.)

main() is a function.
All C programs start by executing from the first statement of the main function.

printf("Welcome to ESC101"); is a statement in C. Statements in C end in semicolon ;

printf

- *printf is the "voice" of the C program*
  - *Used to interact with the users*
- *printf prints its arguments in a certain format*
  - *Format provided by user*

More printing … (code and see)

```
#include <stdio.h>
void main()
{
    printf ("Hello, World! ") ;
    printf ("Hello \n World! \n") ;
}
```

Some more printing

```
#include <stdio.h>
void main()
{
    printf ("Hello, World! \n") ;
    printf ("Hello \n World! \n") ;
    printf ("Hell\no \t World! \n") ;
}
```

Another Simple Program

- Program to add two integers (17 and 23)

```
# include <stdio.h>
int main ()  {
    int a = 17;
    int b = 23;
    int c;
    c = a + b;
    printf("Result is %d", c);
    return 0;
}
```

e program prints the message: **Result is 40**

```
# include <stdio.h>
int main () {
   int a = 17;
   int b = 23;
   int c;
   c = a + b;

   printf "Result is %d",c);
   return 0;
}
```

This tells the compiler to reserve a "box" large enough to hold an integer value. The box is named "a" for use in the rest of the program.

"= 17" stores value 17 in the box that we have named "a".
It is OK to skip this part and store value later as we do for box named "c".

+ is an operator used to add two numbers. The numbers come from the values stored in the boxes named "a" and "b"

%d tells printf to expect one integer argument whose value is to be printed. We call it placeholder.
We will see more placeholders soon.

printf (% format)

- % format specifiers allow C program to print things whose values are yet not computed
  - will be known while running the program
- %... is similar to the blanks in a lab sheets used for

Gr- 9- IGCSE Marks:- / 10 DATE:- 16th March,2012.

Expt.N0:- 16 Time- 2 Block periods (90 min) student's Name:-

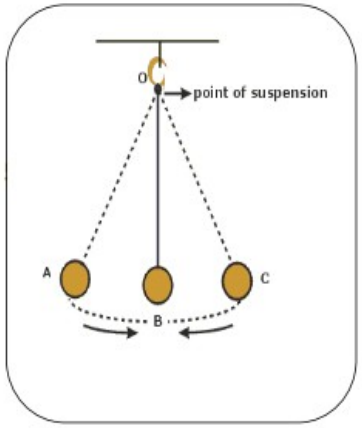
1.Title & Syllabus code:- Simple Pendulum & *General physics- 1.2 + 1.5*

2.Aim:- (i) To prove that the Length of the Simple Pendulum has direct variation with the Time Period for One Oscillation (Period- T second) of the Pendulum.
(ii) Also to understand the graph plot nature of Time PERIOD versus LENGTH.
(iii) Further to confirm that Time Period of Oscillation is Independent of Amplitude.

3.Observation Table:-

Sr. No	Length of Simple Pendulum L / cm	TI ME for 20 Oscillations Trial t 1	Trial t2	t / Second t = (t1 + t 2) / 2	Time for ONE Oscillation - second T = t / 20	PERIOD T / Second With 2 S.F	T^2 / s^2
1	40						
2	60						
3	80						
4	100						
5	120						

4.Diagram:-



5. Formula:-

Time Period (T) is the Time taken for ONE COMPLETE Oscillation of the Simple Pendulum.

B-the Mean / Rest position of BOB.

A or C- Equilibrium Position

$$T = \frac{Total\ Time\ Taken(t)}{Number\ of\ Oscillations}$$

$T = \frac{t}{20}$ in Second

Do you now understand this program?

- Program to add two integers (17 and 23).

```
# include <stdio.h>
int main ()  {
    int a = 17;
    int b = 23;
    int c;
    c = a + b;
    printf("Result is %d", c);
    return 0;
}
```

e program prints the message: **Result is 40**

Reading values from keyboard

```
#include <stdio.h>
void main()
{
        int num ;
        scanf ("%d", &num) ;
        printf (“No. of students is %d\n”, num)
}
```

Centigrade to Fahrenheit

```
#include <stdio.h>
void main()
{
    float cent, fahr;
    scanf("%f",&cent);
    fahr = cent*(9.0/5.0) + 32;
    printf( "%f C equals %f F\n", cent, fahr);
}
```

Next class

- The alphabet of C
  - Like int
- The grammar of C
  - Like using “ ” to delimit strings
- The keywords of C
  - Like printf
- Related reading: Chapter 2.1-2.3

ACKNOWLEDGEMENTS

Slides were inspired from Prof Rajat Moona and Prof. Amey Karkare for the course ESC101 in IIT Kanpur.